

State Transition Diagram, Gaming, and Statistics 3

ME135/ME235 Spring 2024

1 Lab Assignment

Beginning with this lab, you will notice that the assignment may be a bit vaguer than the earlier lab assignments. This is by design. It is so to allow the maximum opportunity for **creativity**. It is also to reflect what is typically in industry. More often than not clients may have ideas of what they want, but do not have the background to define the specifics. It is up to you, being the expert in the field, to develop the proper sets of questions to develop what is actually needed by the problem. I know this is frustrating to some if not all of you. All of us (GSI and myself) are here to answer those questions if needed.

In this lab assignment your group will have a good opportunity to divide and conquer this assignment. It will be too much work for any one individual to complete all of the tasks. **Please do start early**. We will continue developing LabVIEW code in this lab. Moving forward future labs will need to use the ESP32. So, if you have not picked up your ESP32, please do so.

Two programming challenges : (1) Develop a LabVIEW Vi to duplicate a simple version of the Electronic Simon Game utilizing the **timed-loop** and **property nodes**. Here is a short video illustrating the traditional Simon Game.

<https://youtu.be/1Yqj76Q4jI4>

There are many similar videos of the Simon Game that you may use as reference.

(2) develop a LabVIEW VI to illustrate the famous statistic claims on the outcome of the famous Monty Hall's Problem. Monty Hall's "Let's Make a Deal" was a very popular TV game show of the 70's. (No judging) Here is a short video explaining the Monty Hall's Problem.

<https://youtu.be/4Lb-6rxZxx0>

The classic Monty Hall's problem deals with the statistics of choosing the winner out of three doors. On the video, the presenter illustrates a version for one hundred doors as well. Your VI should have the ability to demonstrate up to ten doors to show the statistical claims of choosing the winning door if you were to switch when given the choice after revealing all the losing doors except one. With your VI, show that given the final two unopen doors, your chances of ending up with the winning door is actually 50-50. Statistics works best when you approach infinite trials, with your VI illustrating your odds of winning if you switch every time, you are given the choice as a function of number of times you play the game.

1.1 Deliverables

1. Simon VI
2. Monty Hall's VI

2 Timed Loop

The following section is a review of the timed loop construct in LabVIEW. The timed loop is a special form of the do-while loop with the ability to:

1. Dictate the execution time for the loop.
2. Specify the clock source.
3. Assign the loop to specify processor core.
4. Access to various timing property
5. Ability to change loop parameters from within the loop.
6. Ability to assign priority of execution to the loop.

The timed loop is invoked by selecting the timed loop icon from the structure's palette:

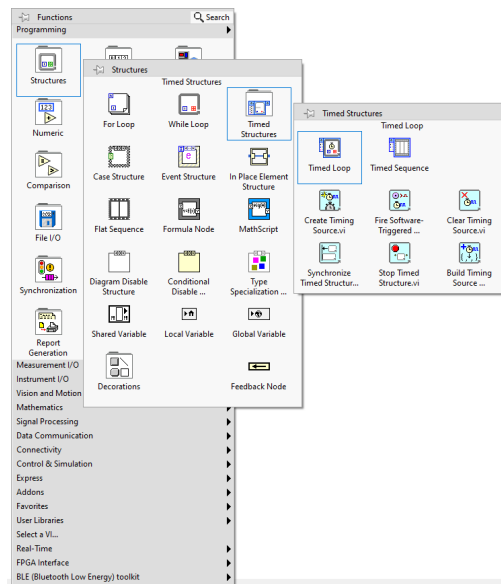


Figure 1.0 Timed Structure Palette



Figure 2.0 Timed Loop

As illustrated in Figure 2.0, the timed loop appears similar to the normal do-while loop. Both loops operate in similar manner. The difference is that the timed loop is feature packed with time related data. You should notice that one difference between a timed loop and a do-while loop is that you no longer need anything connected to the loop stop in a timed loop. Every do-while loop requires that a Boolean source is connected to the loop stop. Although it is still good practice to connect a Boolean source to a timed loop as well, with a timed loop it is not necessary because behind the scenes, the timed loop ensures that Windows gets the processor with every loop cycle. This is not true with the normal do-while loop. The normal do-while loop can monopolize the processor and lock out Windows, making it difficult to stop the loop or the VI if a Boolean condition was not connected to the loop stop.

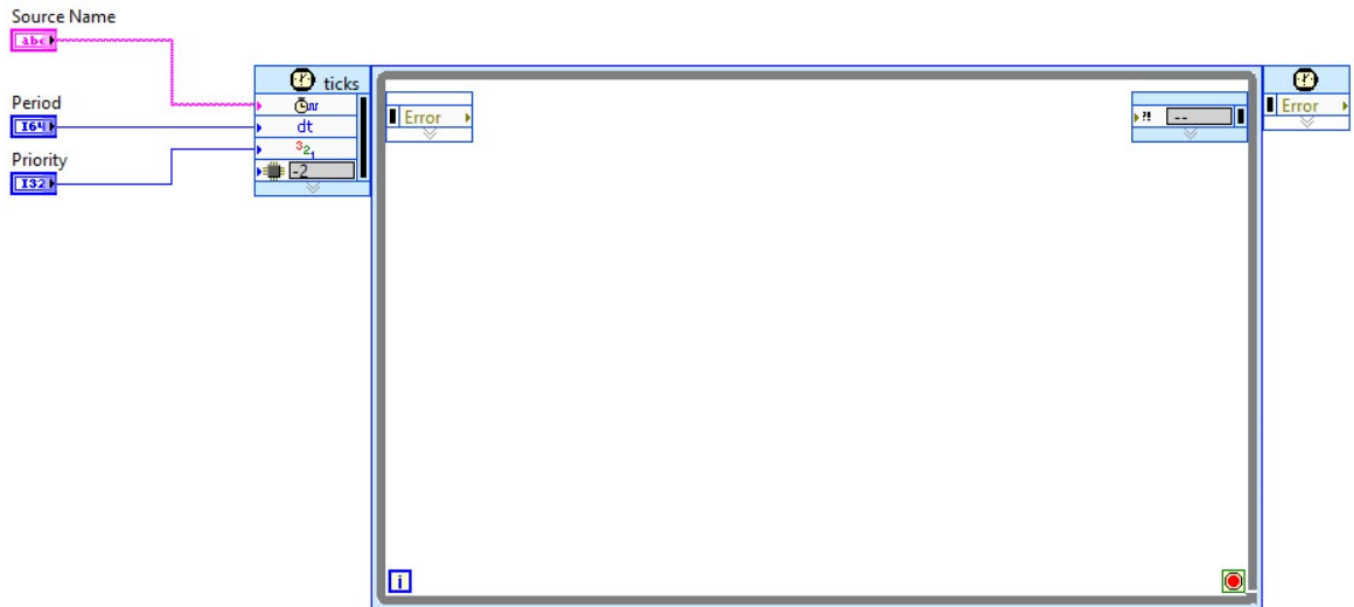


Figure 3.0 Ability to programmatically set loop parameters.

Figure 3.0 illustrates the capability to initially set loop parameters programmatically within the VI. Figure 4.0 illustrates how to set the default settings for the particular timed loop.

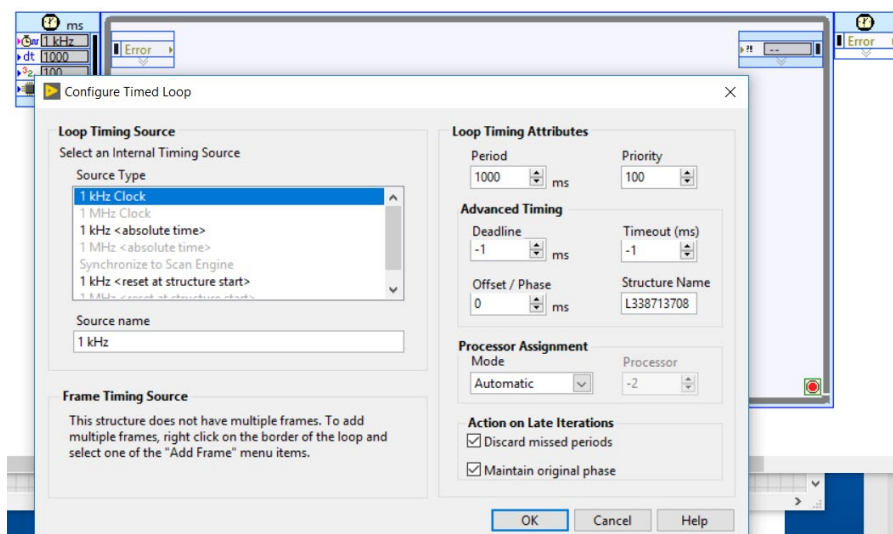


Figure 4.0 Setting default parameters of the timed loop

The manual parameter window is obtained by right clicking the upper left palette attached to the timed loop. One

can set the priority of each timed loop execution by setting the Priority parameter. The higher the value, the higher the priority. Windows by default has a priority of 500. Setting the loop priority above 500 will lead to problematic Window performance. This is highly not recommended. The ability to assign priority to different loops within a VI allows for multitasking.

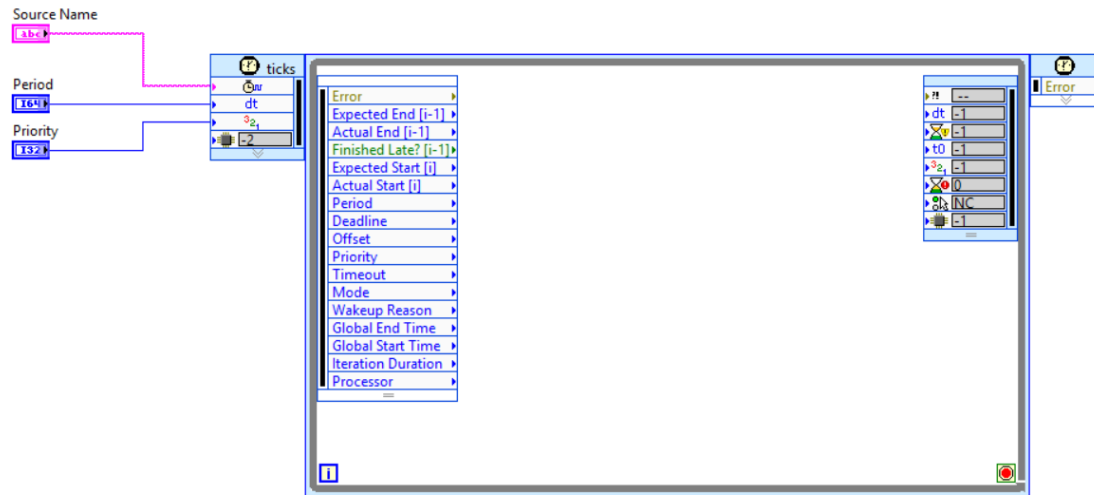


Figure 5.0 Various loop timing data and internal parameter settings

Within the timed loop there are two expandable palettes. The left palette as shown in Figure 5.0 makes available to the VI various parameters and timing data of the time loop during execution. A very useful Boolean is the Finished Late? [i-1] source that can be used as an indicator that the execution in the previous cycle of the timed loop ran late. The right palette allows the internal loop execution to programmatically change the loop parameters, such as loop cycle time.

3 Property Node

Property Nodes allow you to set the properties and get the properties of LabVIEW objects. In some cases, you may want to programmatically change the appearance of the object on your front panel object, this is best done with property nodes of the front panel object. Some examples are to make objects visible or invisible, change colors programmatically, change their size, or even move their location programmatically.

Here is an example of how property nodes may be used.

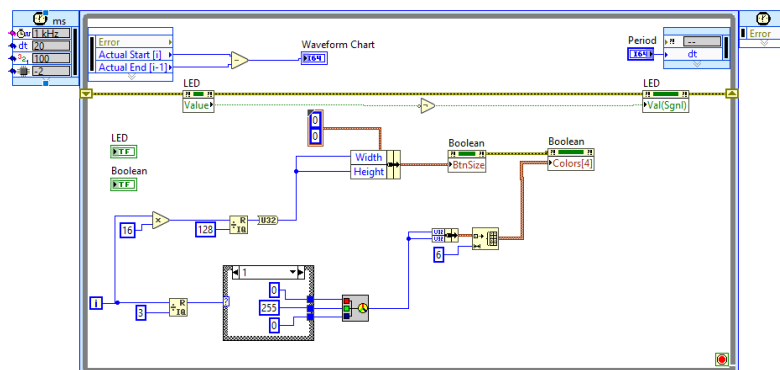


Figure 6.0 Examples of Property Nodes

The property node is created by right clicking on the control or indicator and selecting Create >> Property Node. Once selected all of the property of the control or indicator which are programmatically changed will appear. Any of the property can be selected. Figure 8.0 illustrates an example of the property list. Figure 9.0 is an example of the button size property and button colors property for the Boolean indicator in Figure 7.0.

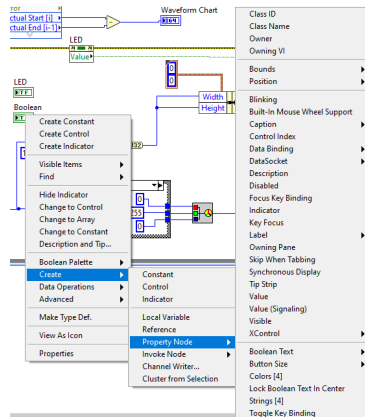


Figure 8.0 Creating a property node.

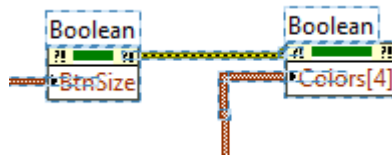


Figure 9.0 Example of Button Size and Colors property

Property nodes can be read from and written to.